Código de Juego del Ahorcado

Grupo L — Sistema de Gestión Taller Mecánico | C++

SECCIÓN 1 — Código fuente

main.cpp 2

main.cpp > ...

```cpp
#include <iostream>
#include <string>
#include <cstdlib>
#include <ctime>
#include <cctype>
using namespace std;

// ==================== CLASE REPUESTO ====================
class Repuesto {
private:
    string codigo;
    string nombre;
    double precioUnitario;
    int cantidad;

public:
    Repuesto() {
        codigo = "";
        nombre = "";
        precioUnitario = 0.0;
        cantidad = 0;
    }

    void setCodigo(string c)    { codigo = c; }
    void setNombre(string n)    { nombre = n; }
    void setPrecio(double p)    { precioUnitario = p; }
    void setCantidad(int c)     { cantidad = c; }

    string getCodigo()          { return codigo; }
    string getNombre()          { return nombre; }
    double getPrecio()          { return precioUnitario; }
    int getCantidad()           { return cantidad; }
};

// ==================== CLASE ORDENTALLER ====================
class OrdenTaller {
private:
```

```
// ==================== CLASE ORDENTALLER ====================
class OrdenTaller {
private:
    string placa;
    string cliente;
    string cedula;
    string tipoServicio;
    double horasManoObra;
    double costoPorHora;
    string estado;
    string repuestosUsados[5];
    int cantidadRepuestos[5];
    int numRepuestos;

public:
    OrdenTaller() {
        placa = "";
        cliente = "";
        cedula = "";
        tipoServicio = "";
        horasManoObra = 0.0;
        costoPorHora = 0.0;
        estado = "pendiente";
        numRepuestos = 0;
    }

    void setPlaca(string p)         { placa = p; }
    void setCliente(string c)       { cliente = c; }
    void setCedula(string c)        { cedula = c; }
    void setTipoServicio(string t)  { tipoServicio = t; }
    void setHoras(double h)         { horasManoObra = h; }
    void setCostoPorHora(double c)  { costoPorHora = c; }
    void setEstado(string e)        { estado = e; }
```

```
    void agregarRepuesto(string nombre, int cant) {
        if (numRepuestos < 5) {
            repuestosUsados[numRepuestos] = nombre;
            cantidadRepuestos[numRepuestos] = cant;
            numRepuestos++;
        }
    }

    double getCostoManoObra() {
        return horasManoObra * costoPorHora;
    }

    double getCostoRepuestos(Repuesto repuestos[], int numRep) {
        double total = 0;
        for (int i = 0; i < numRepuestos; i++) {
            for (int j = 0; j < numRep; j++) {
                if (repuestosUsados[i] == repuestos[j].getNombre()) {
                    total += repuestos[j].getPrecio() * cantidadRepuestos[i];
                }
            }
        }
        return total;
    }

    double getValorFinal(Repuesto repuestos[], int numRep) {
        double subtotal = getCostoManoObra() + getCostoRepuestos(repuestos, numRep);
        double descuento = 0;
        if (tipoServicio == "preventivo") descuento = subtotal * 0.10;
        else if (tipoServicio == "express") descuento = subtotal * 0.05;
        double impuesto = (subtotal - descuento) * 0.15;
        return subtotal - descuento + impuesto;
    }

    string getClasificacion(Repuesto repuestos[], int numRep) {
```

```
// ==================== ARREGLOS GLOBALES ====================
const int MAX = 50;
Repuesto repuestos[MAX];
OrdenTaller ordenes[MAX];
int numRepuestos = 0;
int numOrdenes = 0;
```

```
void registrarRepuesto() {
    if (numRepuestos >= MAX) {
        cout << "\nNo hay espacio para mas repuestos." << endl;
        return;
    }
    string codigo, nombre;
    double precio;
    int cantidad;
    cout << "\n===== REGISTRAR REPUESTO =====" << endl;
    cout << "Codigo: ";
    cin >> codigo;
    for (int i = 0; i < numRepuestos; i++) {
        if (repuestos[i].getCodigo() == codigo) {
            cout << "Ya existe un repuesto con ese codigo." << endl;
            return;
        }
    }
    cout << "Nombre: ";
    cin.ignore();
    getline(cin, nombre);
    do {
        cout << "Precio unitario: ";
        cin >> precio;
        if (precio <= 0) cout << "El precio debe ser mayor a 0." << endl;
    } while (precio <= 0);
    do {
        cout << "Cantidad en stock: ";
        cin >> cantidad;
        if (cantidad < 0) cout << "La cantidad no puede ser negativa." << endl;
    } while (cantidad < 0);
    repuestos[numRepuestos].setCodigo(codigo);
    repuestos[numRepuestos].setNombre(nombre);
    repuestos[numRepuestos].setPrecio(precio);
    repuestos[numRepuestos].setCantidad(cantidad);
    numRepuestos++;
    cout << "Repuesto registrado correctamente." << endl;
}
```

```
// ==================== REGISTRAR ORDEN ====================
void registrarOrden() {
    if (numOrdenes >= MAX) {
        cout << "\nNo hay espacio para mas ordenes." << endl;
        return;
    }
    string placa, cliente, cedula, tipoServicio;
    double horas, costoPorHora;
    cout << "\n===== REGISTRAR ORDEN DE TRABAJO =====" << endl;
    cout << "Placa del vehiculo: ";
    cin >> placa;
    cout << "Nombre del cliente: ";
    cin.ignore();
    getline(cin, cliente);
    cout << "Cedula del cliente: ";
    cin >> cedula;
    cout << "Tipo de servicio (preventivo/correctivo/express): ";
    cin >> tipoServicio;
    while (tipoServicio != "preventivo" && tipoServicio != "correctivo" && tipoServicio != "express") {
        cout << "Tipo invalido. Ingrese preventivo, correctivo o express: ";
        cin >> tipoServicio;
    }
    do {
        cout << "Horas de mano de obra: ";
        cin >> horas;
        if (horas <= 0) cout << "Las horas deben ser mayor a 0." << endl;
    } while (horas <= 0);
    do {
        cout << "Costo por hora: ";
        cin >> costoPorHora;
        if (costoPorHora <= 0) cout << "El costo debe ser mayor a 0." << endl;
    } while (costoPorHora <= 0);
    ordenes[numOrdenes].setPlaca(placa);
    ordenes[numOrdenes].setCliente(cliente);
    ordenes[numOrdenes].setCedula(cedula);
    ordenes[numOrdenes].setTipoServicio(tipoServicio);
```

```cpp
// ==================== MOSTRAR ORDEN ====================
void mostrarOrden(int i) {
    cout << "\n--- Orden #" << i+1 << " ---" << endl;
    cout << "Placa:           " << ordenes[i].getPlaca() << endl;
    cout << "Cliente:         " << ordenes[i].getCliente() << endl;
    cout << "Cedula:          " << ordenes[i].getCedula() << endl;
    cout << "Tipo servicio:   " << ordenes[i].getTipoServicio() << endl;
    cout << "Horas:           " << ordenes[i].getHoras() << endl;
    cout << "Costo/hora:      $" << ordenes[i].getCostoPorHora() << endl;
    cout << "Estado:          " << ordenes[i].getEstado() << endl;
    cout << "Repuestos usados:" << endl;
    for (int j = 0; j < ordenes[i].getNumRepuestos(); j++) {
        cout << "  - " << ordenes[i].getRepuestoNombre(j)
             << " x" << ordenes[i].getRepuestoCantidad(j) << endl;
    }
    cout << "Costo mano obra: $" << ordenes[i].getCostoManoObra() << endl;
    cout << "Costo repuestos: $" << ordenes[i].getCostoRepuestos(repuestos, numRepuestos) << endl;
    cout << "Valor final:     $" << ordenes[i].getValorFinal(repuestos, numRepuestos) << endl;
    cout << "Clasificacion:   " << ordenes[i].getClasificacion(repuestos, numRepuestos) << endl;
}
```

```cpp
// ==================== BUSCAR ORDEN ====================
void buscarOrden() {
    int opcion;
    cout << "\n===== BUSCAR ORDEN =====" << endl;
    cout << "1. Buscar por placa" << endl;
    cout << "2. Buscar por cedula" << endl;
    cout << "Opcion: ";
    cin >> opcion;
    string busqueda;
    bool encontrado = false;
    if (opcion == 1) {
        cout << "Ingrese la placa: ";
        cin >> busqueda;
        for (int i = 0; i < numOrdenes; i++) {
            if (ordenes[i].getPlaca() == busqueda) {
                mostrarOrden(i);
                encontrado = true;
            }
        }
    } else if (opcion == 2) {
        cout << "Ingrese la cedula: ";
        cin >> busqueda;
        for (int i = 0; i < numOrdenes; i++) {
            if (ordenes[i].getCedula() == busqueda) {
                mostrarOrden(i);
                encontrado = true;
            }
        }
    } else {
        cout << "Opcion invalida." << endl;
        return;
    }
    if (!encontrado) cout << "No se encontraron ordenes." << endl;
}
```

```cpp
// ==================== ORDENAR ORDENES ====================
void ordenarOrdenes() {
    if (numOrdenes < 2) {
        cout << "\nNo hay suficientes ordenes para ordenar." << endl;
        return;
    }
    for (int i = 0; i < numOrdenes - 1; i++) {
        for (int j = 0; j < numOrdenes - 1 - i; j++) {
            double valorA = ordenes[j].getValorFinal(repuestos, numRepuestos);
            double valorB = ordenes[j+1].getValorFinal(repuestos, numRepuestos);
            if (valorA < valorB) {
                OrdenTaller temp = ordenes[j];
                ordenes[j] = ordenes[j+1];
                ordenes[j+1] = temp;
            }
        }
    }
    cout << "\nOrdenes ordenadas por mayor costo:" << endl;
    for (int i = 0; i < numOrdenes; i++) {
        cout << i+1 << ". " << ordenes[i].getPlaca()
             << " - " << ordenes[i].getCliente()
             << " - $" << ordenes[i].getValorFinal(repuestos, numRepuestos)
             << " - " << ordenes[i].getClasificacion(repuestos, numRepuestos)
             << endl;
    }
}
```

```cpp
// ==================== VER HISTORIAL ====================
void verHistorial() {
    if (numOrdenes == 0) {
        cout << "\nNo hay ordenes registradas." << endl;
        return;
    }
    cout << "\n===== HISTORIAL DE ORDENES =====" << endl;
    for (int i = 0; i < numOrdenes; i++) {
        mostrarOrden(i);
    }
    cout << "\n--- Ordenes con baja rentabilidad ---" << endl;
    bool hayBaja = false;
    for (int i = 0; i < numOrdenes; i++) {
        if (ordenes[i].getValorFinal(repuestos, numRepuestos) < 100) {
            cout << "  Placa: " << ordenes[i].getPlaca()
                 << " - Cliente: " << ordenes[i].getCliente()
                 << " - Valor: $" << ordenes[i].getValorFinal(repuestos, numRepuestos)
                 << endl;
            hayBaja = true;
        }
    }
    if (!hayBaja) cout << "  Ninguna orden con baja rentabilidad." << endl;
}
```

```
// ==================== FRAGMENTO CORREGIDO ====================
// Errores corregidos:
// 1. Falta punto y coma en linea del impuesto
// 2. Retornaba manoObra en lugar de total
// 3. No validaba que descuento no supere el subtotal
double totalOrden(double manoObra, double repuestos, double descuento) {
    double subtotal = manoObra + repuestos;
    if (descuento > subtotal) descuento = subtotal;
    double total = subtotal - descuento;
    total = total + total * 0.15;
    if (total < 0) total = 0;
    return total;
}

// ==================== FUNCION RECURSIVA ====================
double sumarTotalOrdenes(int indice) {
    if (indice >= numOrdenes) return 0;
    return ordenes[indice].getValorFinal(repuestos, numRepuestos)
            + sumarTotalOrdenes(indice + 1);
}
```

```
// ==================== MODULO AHORCADO ====================
string palabrasFacil[] = {"ORDEN", "PLACA", "MOTOR", "FRENO", "LLANTA"};
string palabrasDificil[] = {"CARBURADOR", "TRANSMISION", "AMORTIGUADOR", "DIFERENCIAL", "ALTERNADOR"};
string historialGanador[20];
string historialPalabra[20];
int historialPartidas = 0;

void mostrarAhorcado(int errores) {
    cout << "\n";
    switch(errores) {
        case 0: cout << "  +---+\n  |   |\n      |\n      |\n      |\n      |\n  =======\n"; break;
        case 1: cout << "  +---+\n  |   |\n  O   |\n      |\n      |\n      |\n  =======\n"; break;
        case 2: cout << "  +---+\n  |   |\n  O   |\n  |   |\n      |\n      |\n  =========\n"; break;
        case 3: cout << "  +---+\n  |   |\n  O   |\n /|   |\n      |\n      |\n  =========\n"; break;
        case 4: cout << "  +---+\n  |   |\n  O   |\n /|\\  |\n      |\n      |\n  =========\n"; break;
        case 5: cout << "  +---+\n  |   |\n  O   |\n /|\\  |\n /    |\n      |\n  =========\n"; break;
        case 6: cout << "  +---+\n  |   |\n  O   |\n /|\\  |\n / \\  |\n      |\n  =========\n"; break;
    }
}
```

```
    }
    mostrarAhorcado(errores);
    string ganador;
    if (gano) {
        cout << "\nFelicidades! Adivinaste la palabra: " << palabra << endl;
        ganador = "Jugador";
    } else {
        cout << "\nPerdiste. La palabra era: " << palabra << endl;
        ganador = "CPU";
    }
    if (historialPartidas < 20) {
        historialGanador[historialPartidas] = ganador;
        historialPalabra[historialPartidas] = palabra;
        historialPartidas++;
    }
}

void verHistorialAhorcado() {
    if (historialPartidas == 0) {
        cout << "\nNo hay partidas jugadas aun." << endl;
        return;
    }
    cout << "\n===== HISTORIAL DE PARTIDAS =====" << endl;
    int jugadorGana = 0, cpuGana = 0;
    for (int i = 0; i < historialPartidas; i++) {
        cout << i+1 << ". Palabra: " << historialPalabra[i]
             << " - Resultado: " << historialGanador[i] << endl;
        if (historialGanador[i] == "Jugador") jugadorGana++;
        else cpuGana++;
    }
    cout << "\nJugador: " << jugadorGana << " victorias" << endl;
    cout << "CPU: " << cpuGana << " victorias" << endl;
}
```

```cpp
// ==================== MENU PRINCIPAL ====================
void menuPrincipal() {
    int opcion;
    do {
        cout << "\n========================================" << endl;
        cout << "   SISTEMA DE GESTION - TALLER MECANICO" << endl;
        cout << "========================================" << endl;
        cout << "  1. Registrar repuesto" << endl;
        cout << "  2. Registrar orden de trabajo" << endl;
        cout << "  3. Buscar orden" << endl;
        cout << "  4. Ordenar ordenes" << endl;
        cout << "  5. Ver historial" << endl;
        cout << "  6. Total facturado (recursivo)" << endl;
        cout << "  7. Modulo Ahorcado" << endl;
        cout << "  0. Salir" << endl;
        cout << "----------------------------------------" << endl;
        cout << "  Opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1: registrarRepuesto(); break;
            case 2: registrarOrden(); break;
            case 3: buscarOrden(); break;
            case 4: ordenarOrdenes(); break;
            case 5: verHistorial(); break;
            case 6: cout << "\nTotal facturado: $" << sumarTotalOrdenes(0) << endl; break;
            case 7: menuAhorcado(); break;
            case 0: cout << "\nCerrando sistema..." << endl; break;
            default: cout << "\nOpcion invalida." << endl;
        }
    } while (opcion != 0);
}
```

```
// ==================== MAIN ====================
int main() {
    menuPrincipal();
    return 0;
}
```

SECCIÓN 2 — Prueba de ejecución del programa

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

========================================
   SISTEMA DE GESTION - TALLER MECANICO
========================================
  1. Registrar repuesto
  2. Registrar orden de trabajo
  3. Buscar orden
  4. Ordenar ordenes
  5. Ver historial
  6. Total facturado (recursivo)
  7. Modulo Ahorcado
  0. Salir
----------------------------------------
  Opcion:
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

  5. Ver historial
  6. Total facturado (recursivo)
  7. Modulo Ahorcado
  0. Salir
----------------------------------------
  Opcion: 7

===== AHORCADO - TALLER MECANICO =====
1. Jugar
2. Ver historial de partidas
3. Ver instrucciones
0. Volver al menu principal
Opcion:
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Tienes 6 intentos.

  +---+
  |   |
      |
      |
      |
      |
=========
Palabra: _ _ _ _ _
Letras usadas:
Intentos restantes: 6
Ingrese una letra:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
0. Volver al menu principal
Opcion: 1

Seleccione nivel:
1. Facil
2. Dificil
Opcion: 1
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

 /|\  |
 / \  |
      |
=========

Perdiste. La palabra era: MOTOR

===== AHORCADO - TALLER MECANICO =====
1. Jugar
2. Ver historial de partidas
3. Ver instrucciones
0. Volver al menu principal
Opcion:
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

  +---+
  |   |
  O   |
 /|\  |
 / \  |
      |
=========

Perdiste. La palabra era: MOTOR
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
===== HISTORIAL DE PARTIDAS =====
1. Palabra: MOTOR - Resultado: CPU

Jugador: 0 victorias
CPU: 1 victorias
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
--- INSTRUCCIONES ---
Adivina la palabra del vocabulario de taller mecanico.
Tienes 6 intentos antes de que el ahorcado este completo.
Ingresa una letra por turno.
Nivel facil: palabras cortas del taller.
Nivel dificil: palabras tecnicas del taller.
```

```
  7. Modulo Ahorcado
  0. Salir
---------------------------------------
  Opcion: 2

===== REGISTRAR ORDEN DE TRABAJO =====
Placa del vehiculo: abcd-123
Nombre del cliente: edy
Cedula del cliente: 0503565335
Tipo de servicio (preventivo/correctivo/express): preventivo
Horas de mano de obra: 4
Costo por hora: 25
Desea agregar repuestos a la orden? (s/n):
```

```
===== HISTORIAL DE ORDENES =====

--- Orden #1 ---
Placa:           ABCD-123
Cliente:         Edison
Cedula:          0503565335
Tipo servicio:   preventivo
Horas:           4
Costo/hora:      $25
Estado:          pendiente
Repuestos usados:
Costo mano obra: $100
Costo repuestos: $0
Valor final:     $103.5
Clasificacion:   Moderada

--- Orden #2 ---
Placa:           abcd-123
Cliente:         edy
Cedula:          0503565335
Tipo servicio:   preventivo
Horas:           4
Costo/hora:      $25
Estado:          pendiente
Repuestos usados:
Costo mano obra: $100
Costo repuestos: $0
Valor final:     $103.5
Clasificacion:   Moderada

--- Ordenes con baja rentabilidad ---
  Ninguna orden con baja rentabilidad.
```

```
----------------------------------------
  Opcion: 6

Total facturado: $207

========================================
   SISTEMA DE GESTION - TALLER MECANICO
========================================
  1. Registrar repuesto
  2. Registrar orden de trabajo
  3. Buscar orden
  4. Ordenar ordenes
  5. Ver historial
  6. Total facturado (recursivo)
  7. Modulo Ahorcado
  0. Salir
----------------------------------------
  Opcion:
```

```
  Opcion: 4

Ordenes ordenadas por mayor costo:
1. ABCD-123 - Edison - $103.5 - Moderada
2. abcd-123 - edy - $103.5 - Moderada

========================================
   SISTEMA DE GESTION - TALLER MECANICO
========================================
  1. Registrar repuesto
  2. Registrar orden de trabajo
  3. Buscar orden
  4. Ordenar ordenes
  5. Ver historial
  6. Total facturado (recursivo)
  7. Modulo Ahorcado
  0. Salir
----------------------------------------
  Opcion:
```

```
Ordenes ordenadas por mayor costo:
1. ABCD-123 - Edison - $103.5 - Moderada
2. abcd-123 - edy - $103.5 - Moderada

========================================
   SISTEMA DE GESTION - TALLER MECANICO
========================================
  1. Registrar repuesto
  2. Registrar orden de trabajo
  3. Buscar orden
  4. Ordenar ordenes
  5. Ver historial
  6. Total facturado (recursivo)
  7. Modulo Ahorcado
  0. Salir
----------------------------------------
  Opcion: 0
```

```
========================================
   SISTEMA DE GESTION - TALLER MECANICO
========================================
  1. Registrar repuesto
  2. Registrar orden de trabajo
  3. Buscar orden
  4. Ordenar ordenes
  5. Ver historial
  6. Total facturado (recursivo)
  7. Modulo Ahorcado
  0. Salir
----------------------------------------
  Opcion: 4

Ordenes ordenadas por mayor costo:
1. ABCD-123 - Edison - $103.5 - Moderada
2. abcd-123 - edy - $103.5 - Moderada

========================================
   SISTEMA DE GESTION - TALLER MECANICO
========================================
  1. Registrar repuesto
  2. Registrar orden de trabajo
  3. Buscar orden
  4. Ordenar ordenes
  5. Ver historial
  6. Total facturado (recursivo)
  7. Modulo Ahorcado
  0. Salir
----------------------------------------
  Opcion: 0

Cerrando sistema...
```